

BACKEND INTERVIEW GUIDE - Complete

Spring Boot, Java 21, Microservices & Security *Date: 2026-09-17 | LMS Project*

TABLE OF CONTENTS

1. Spring Boot Configuration & Architecture
 2. REST APIs & HTTP
 3. Database Design (DynamoDB)
 4. Security & Authentication
 5. Multi-Tenancy Patterns
 6. Caching & Performance
 7. Async Processing & Resilience
 8. EventBridge & Messaging
 9. Interview Scenarios & Q&A
-

SECTION 1: SPRING BOOT CONFIGURATION

Application Entry Point

```
@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})
@EnableCaching
@EnableWebSecurity
@EnableMethodSecurity(securedEnabled = true, prePostEnabled = true, jsr250Enabled = true)
public class LmsUserServiceApplication {

    @Bean
    public WebMvcConfigurer corsConfigurer() {
        registry.addMapping("/api/v1/**")
            .allowedOriginPatterns(corsUrl.split(","))
            .allowedMethods("*")
            .allowCredentials(true)
            .allowedHeaders("*");
    }

    public static void main(String[] args) {
        SpringApplication.run(LmsUserServiceApplication.class, args);
    }
}
```

Why Exclude DataSourceAutoConfiguration?

EDS (Enterprise Data Service) is optional feature. If disabled:

- No JDBC datasource configured

- Application starts without database
- DataSource built manually only when `eds.enabled=true`

SECTION 2: REST APIS & HTTP

Endpoint Patterns

Base: `/api/v1/`

User Service:

GET	<code>/api/v1/users/search</code>	- Search with pagination
POST	<code>/api/v1/users/upload</code>	- Bulk user creation
PUT	<code>/api/v1/users/update</code>	- Update user
POST	<code>/api/v1/users/deactivate</code>	- Deactivate account

Course Service:

GET	<code>/api/v1/courses</code>	- List courses
GET	<code>/api/v1/courses/{courseId}</code>	- Course details
POST	<code>/api/v1/enrollments</code>	- Enroll in course
DELETE	<code>/api/v1/enrollments/{id}</code>	- Unenroll

CORS Configuration

```
.allowedOriginPatterns(corsUrl.split(",")) // *.lms.com
.allowedHeaders("Content-Type", "Authorization", "X-Tenant-ID",
                "X-Refresh-Token", "X-Cognito-Username")
.allowedMethods("GET", "DELETE", "POST", "PUT", "OPTIONS", "PATCH")
.allowCredentials(true); // Allow cookies + auth headers
```

Response Format

```
{
  "status": 200,
  "data": { /* entity */ },
  "error": null
}
```

SECTION 3: DATABASE DESIGN - DYNAMODB

Why DynamoDB Over Relational?

Aspect	DynamoDB	PostgreSQL
Scaling	Automatic partitioning	Manual sharding
Tenancy	Natural via partition key	Need filters
Latency	Single-digit ms	Depends on complexity
Cost	Pay per request	Fixed servers
Flexibility	Sparse attributes	Strict schema

This system uses: DynamoDB primarily + optional PostgreSQL (EDS for complex queries)

Single-Table Design

Partition Key (pk): tenantCode → Tenant isolation
Sort Key (sk): entityType#entityId → Multiple entity types

Example:

pk	sk	type	firstName
TENANT#123	USER#user-456	USER	John
TENANT#123	COURSE#course-1	COURSE	Math
TENANT#123	ENROLLMENT#e-1	ENROLL	null
TENANT#456	USER#user-789	USER	Jane

Benefits:

- Single transaction for multi-entity consistency
- Natural tenant isolation (different pk = no cross-tenant access)
- Efficient pagination (cursor includes pk + sk)

Global Secondary Index (GSI):

GSI on (type): Supports "get all users in tenant" queries

DynamoDB Queries

```
// Read with tenant scoping
QueryRequest query = QueryRequest.builder()
    .tableName("lms-table")
    .keyConditionExpression("pk = :pk AND sk BEGINS_WITH :sk")
    .expressionAttributeValues(Map.of(
        ":pk", AttributeValue.builder()
```

```

        .s(TenantUtil.getTenantCode()) // ← Tenant-scoped
        .build(),
        ":sk", AttributeValue.builder()
        .s("USER#") // Only users
        .build()
    ))
    .build();

List<User> users = dynamoDbClient.query(query).items();

```

Query Patterns:

- Partition Key (required) + Sort Key (optional range)
- FilterExpression (client-side filtering)
- GSI for alternative access patterns
- Batch operations: BatchGetItem, BatchWriteItem
- Transactions: TransactWriteItems (atomic multi-item)

Pagination

```

// First request
QueryRequest.builder()
    .keyConditionExpression("pk = :pk")
    .limit(10) // 10 items per page
    .build();

// Response includes lastEvaluatedKey
{
    "items": [...10 users...],
    "lastEvaluatedKey": {
        "pk": "TENANT#123",
        "sk": "USER#user-456"
    }
}

// Next request
QueryRequest.builder()
    .keyConditionExpression("pk = :pk")
    .limit(10)
    .exclusiveStartKey(lastEvaluatedKey) // Cursor
    .build();

```

Why cursor-based (not offset)?

- O(1) lookup (direct start point)
- vs O(n) offset (must scan to position)
- More efficient for large datasets

Throughput Management

```
// 1. Exponential Backoff
for (int attempt = 0; attempt < 5; attempt++) {
    try {
        return dynamoDbClient.scan(request);
    } catch (ProvisionedThroughputExceededException e) {
        long backoff = 100 * (1L << attempt); // 100ms, 200ms, 400ms...
        Thread.sleep(backoff);
    }
}

// 2. Rate Limiting (Resilience4j)
RateLimiterConfig config = RateLimiterConfig.custom()
    .timeoutDuration(Duration.ofSeconds(60))
    .limitRefreshPeriod(Duration.ofSeconds(1))
    .limitForPeriod(100) // Max 100 calls/second
    .build();

// 3. Batch Operations
List<User> users = new ArrayList<>();
for (int i = 0; i < 1000; i++) { // Read 1000 (even with filter)
    users.addAll(query(...));
}
// DynamoDB applies filter after limit
// Client-side pagination: slice for page display

// 4. Billing Mode
// PAY_PER_REQUEST: Auto-scales, no tuning needed
// vs PROVISIONED: Manual WCU/RCU
```

SECTION 4: SECURITY & AUTHENTICATION

JWT Authentication Flow

Lambda Authorizer (API Gateway):

```
// 1. Extract token from Authorization header
const token = event.authorizationToken.substring(7); // Remove "Bearer "

// 2. Resolve tenant from Origin header
const host = new URL(event.headers.Origin).hostname;

// 3. Load tenant-specific Cognito pool (S3 cached)
const tenantConfig = await getTenantConfig(host);

// 4. Verify JWT signature against tenant's pool
const verifier = CognitoJwtVerifier.create({
    userPoolId: tenantConfig.userPoolId,
```

```

    clientId: tenantConfig.clientId
  });
  const claims = await verifier.verify(token);

  // 5. Return Allow policy with tenant context
  return {
    principalId: claims.sub,
    policyDocument: { Effect: "Allow", Action: "execute-api:Invoke", ... },
    context: { tenantId: tenantCode }
  };

```

Backend JWT Filter:

```

@Slf4j
public class JWTAuthenticationFilter extends BasicAuthenticationFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws IOException {

        String token = extractBearerToken(request);

        // Verify JWT
        Jws<Claims> jws = Jwts.parserBuilder()
            .setSigningKeyResolver(signingKeyResolver)
            .build()
            .parseClaimsJws(token);

        Claims claims = jws.getBody();
        String userId = claims.getSubject();
        List<String> roles = (List<String>) claims.get("cognito:groups");

        // Set security context
        UsernamePasswordAuthenticationToken auth =
            new UsernamePasswordAuthenticationToken(userId, null, authorities);
        SecurityContextHolder.getContext().setAuthentication(auth);

        // Set tenant for DAOs
        String tenantCode = extractTenantFromClaims(claims);
        TenantUtil.setTenant(tenantCode);

        filterChain.doFilter(request, response);
    }
}

```

Spring Security Configuration

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    protected SecurityFilterChain configure(HttpSecurity http) throws Exception {
        return http
            .csrf(AbstractHttpConfigurer::disable) // API, not forms
            .authorizeHttpRequests(requests -> requests
                .requestMatchers("/actuator/health").permitAll()
                .anyRequest().authenticated())
            .addFilter(jwtAuthenticationFilter)
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)) // No
cookies
            .cors(cors -> cors.configure(http))
            .headers(headers -> headers
                // Security headers
                .httpStrictTransportSecurity(hsts -> hsts
                    .includeSubDomains(true)
                    .preload(true)
                    .maxAgeInSeconds(31536000)) // 1 year
                .contentSecurityPolicy(csp -> csp
                    .policyDirectives("default-src 'self'; script-src 'self'; object-src
'none'")))
                .frameOptions(HeadersConfigurer.FrameOptionsConfig::sameOrigin)
                .xssProtection(xss ->
xss.headerValue(XXssProtectionHeaderWriter.HeaderValue.ENABLED_MODE_BLOCK))
                .addHeaderWriter((request, response) -> response.setHeader("Server",
"")))
            .build();
    }
}

```

@PreAuthorize for Method Security

```

@Service
@Slf4j
public class UserServiceImpl {

    @PreAuthorize("hasAnyRole('SYSTEM_ADMIN', 'CONTENT_AUTHOR')")
    public ResponseEntity deleteUser(String userId) {
        // Only admins + authors can delete
        userDao.delete(userId);
        return ResponseEntity.ok("Deleted");
    }

    @PreAuthorize("isAuthenticated()")
    public ResponseEntity getUserProfile() {

```

```
// Any authenticated user
return ResponseEntity.ok(userDao.get());
}

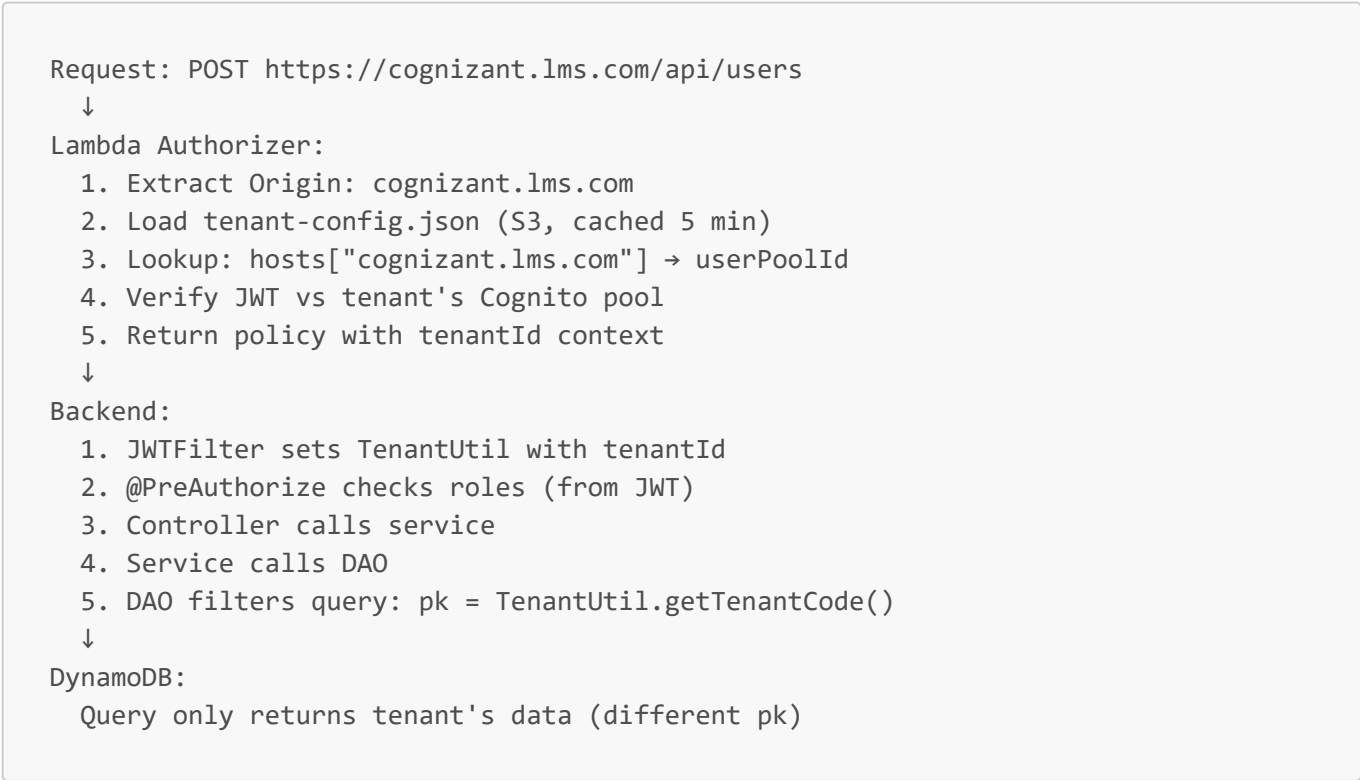
@PreAuthorize("#userId == authentication.principal.name")
public ResponseEntity getOwnProfile(String userId) {
    // Can only access own profile
    return ResponseEntity.ok(userDao.get(userId));
}
}
```

Security Headers Explained

Header	Purpose	Value
HSTS	Force HTTPS	max-age=31536000; includeSubDomains; preload
CSP	Restrict scripts	default-src 'self'; script-src 'self'
X-Frame-Options	Clickjacking	SAMEORIGIN
X-XSS-Protection	XSS detection	1; mode=block
Server	Hide implementation	(removed)

SECTION 5: MULTI-TENANCY PATTERNS

Tenant Resolution Flow



TenantUtil - ThreadLocal Storage

```
@Slf4j
public class TenantUtil {
    private static ThreadLocal<TenantDTO> tenantContext = new ThreadLocal<>();

    public static void setTenant(TenantDTO tenant) {
        tenantContext.set(tenant);
    }

    public static String getTenantCode() {
        TenantDTO tenant = tenantContext.get();
        if (tenant == null) {
            throw new IllegalStateException("Tenant not set in context");
        }
        return tenant.getTenantCode();
    }

    public static void clear() {
        tenantContext.remove(); // Clean up after request
    }
}

// DAO Usage
public class UserDaoImpl implements UserDao {
    public User findById(String userId) {
        QueryRequest query = QueryRequest.builder()
            .keyConditionExpression("pk = :pk AND sk = :sk")
            .expressionAttributeValues(Map.of(
                ":pk", AttributeValue.builder()
                    .s(TenantUtil.getTenantCode()) // Automatic scoping
                    .build(),
                ":sk", AttributeValue.builder()
                    .s("USER#" + userId)
                    .build()
            ))
            .build();

        return dynamoDbClient.query(query).items().stream()
            .findFirst()
            .orElse(null);
    }
}
```

SECTION 6: CACHING & PERFORMANCE

Caffeine In-Memory Caching

```

@Configuration
public class CacheConfig {

    @Bean
    public CacheManager cacheManager() {
        SimpleCacheManager manager = new SimpleCacheManager();
        manager.setCaches(Arrays.asList(
            new CaffeineCache(
                "landingPageStaticData",
                Caffeine.newBuilder()
                    .expireAfterWrite(30, TimeUnit.MINUTES)
                    .maximumSize(500)
                    .build()
            ),
            new CaffeineCache(
                "appStoreData",
                Caffeine.newBuilder()
                    .expireAfterWrite(1, TimeUnit.MINUTES)
                    .maximumSize(200)
                    .build()
            )
        ));
        return manager;
    }
}

```

Caching Usage

```

@Service
public class CategoryServiceImpl {

    @Cacheable(value = "landingPageStaticData", key = "#tenantCode")
    public List<Category> getLandingCategories(String tenantCode) {
        // Query DB only first time (or after TTL expires)
        return categoryDao.getAll(tenantCode);
    }

    @CacheEvict(value = "landingPageStaticData", allEntries = true)
    public void updateCategory(Category category) {
        categoryDao.update(category);
        // Cache cleared for all entries
    }
}

```

When to Cache

✓ Cache if:

- Read-heavy (100+ reads vs 1 write)

- Change infrequently (<daily)
- Expensive computation (sorting, aggregation)
- Reference data (categories, cities, training rooms)

X Don't cache:

- User-specific data (profile)
- Frequently changing (inventory)
- Privacy-sensitive (medical records)

SECTION 7: ASYNC PROCESSING

Async Cognito Client

```
@Bean
public CognitoIdentityProviderAsyncClient cognitoAsyncClient() {
    SdkAsyncHttpClient nettyClient = NettyNioAsyncHttpClient.builder()
        .maxConcurrency(100)
        .connectionAcquisitionTimeout(Duration.ofSeconds(60))
        .build();

    return CognitoIdentityProviderAsyncClient.builder()
        .httpClient(nettyClient)
        .region(Region.of(awsRegion))
        .build();
}
```

CompletableFuture Pattern

```
@Service
public class UserServiceImpl {

    public CompletableFuture<AdminCreateUserResponse> createUserAsync(User user) {
        AdminCreateUserRequest request = AdminCreateUserRequest.builder()
            .userPoolId(userPoolId)
            .username(user.getEmailId())
            .temporaryPassword(generatePassword())
            .messageAction(MessageActionType.SUPPRESS)
            .build();

        return cognitoAsyncClient.adminCreateUser(request)
            .exceptionally(ex -> {
                log.error("Cognito create user failed", ex);
                throw new CognitoServiceException("User creation failed", ex);
            });
    }

    public CompletableFuture<List<User>> importUsersAsync(List<User> users) {
```

```

        List<CompletableFuture<AdminCreateUserResponse>> futures = users
            .stream()
            .map(this::createUserAsync)
            .collect(Collectors.toList());

        return CompletableFuture.allOf(futures.toArray(new CompletableFuture[0]))
            .thenApply(v -> users); // Return after all complete
    }
}

// Controller usage
@PostMapping("/users/import")
public CompletableFuture<ResponseEntity<String>> importUsers(@RequestBody
List<User> users) {
    return userService.importUsersAsync(users)
        .thenApply(result -> ResponseEntity.ok("Imported " + result.size()));
}

```

Why async?

- Cognito calls take 100-500ms
- 10 users synchronously = 1-5 seconds
- 10 users async = 100-500ms (all in parallel)
- Non-blocking (thread not wasted waiting)

SECTION 8: EVENTBRIDGE & MESSAGING

EventBridge Event Publishing

```

@Service
@Slf4j
public class LmsEventBridgePublisherServiceImpl {

    private final EventBridgeClient eventBridgeClient;
    private final String courseManagementEventBusArn;

    public void triggerCoursePublishEvent(String courseId, String eventType) {
        try {
            PutEventsResponse response = eventBridgeClient.putEvents(
                PutEventsRequest.builder()
                    .entries(PutEventsRequestEntry.builder()
                        .source("course.management")
                        .detailType("CoursePublished")
                        .detail(objectMapper.writeValueAsString(Map.of(
                            "courseId", courseId,
                            "eventType", eventType,
                            "timestamp", Instant.now()
                        )))
                    .eventBusName(courseManagementEventBusArn)
                    .build())
            );
        } catch (Exception e) {
            log.error("Failed to publish event: {}", e.getMessage());
        }
    }
}

```

```

        .build()
    );

    if (response.failedEntryCount() > 0) {
        log.error("EventBridge publish failed: {}", response.failedEntryCount());
    }
} catch (Exception e) {
    log.error("Error publishing event", e);
    throw new RuntimeException(e);
}
}
}

```

Cascade Events Pattern

```

Lesson Published Event
↓
Triggers: CourseEnrollmentListener
├─ Updates enrollment records
├─ Publishes: EnrollmentCascade
│   ↓
│   Triggers: EmailService
│       └─ Send completion emails
│
│   Triggers: LeaderboardService
│       └─ Update rankings
│
│   Triggers: AnalyticsService
│       └─ Record analytics

```

SECTION 9: EXCEPTION HANDLING

Custom Exception Hierarchy

```

public class DataBaseException extends RuntimeException { }
public class CognitoServiceException extends RuntimeException { }
public class FileProcessingException extends RuntimeException { }
public class BannerManagementActiveLimitException extends RuntimeException { }

// Usage
try {
    user = userDao.get(userId);
} catch (DataBaseException e) {
    log.error("Database error", e);
    return ResponseEntity.status(500).body("DB error");
}

```

Error Response

```
{
  "status": 500,
  "data": null,
  "error": {
    "code": "DB_ERROR",
    "message": "Failed to fetch user",
    "details": { ... }
  }
}
```

SECTION 10: INTERVIEW SCENARIOS

Q: Walk through an Enroll-in-Course request

Frontend:

```
POST /api/enrollments
Authorization: Bearer <JWT>
X-Tenant-ID: cognizant
{ courseId: "course-123", userId: "user-456" }
```

API Gateway:

- Lambda Authorizer validates JWT vs cognizant Cognito pool
- Returns Allow policy

Backend JWTFilter:

- Extract token, parse claims
- Set SecurityContext (userId, roles)
- Set TenantUtil = "TENANT#cognizant"

EnrollmentController:

```
@PostMapping("/enrollments")
@PreAuthorize("isAuthenticated()")
public ResponseEntity enroll(@RequestBody EnrollmentRequest req) {
    return ResponseEntity.status(201).body(enrollmentService.enroll(req));
}
```

EnrollmentService:

```

public Enrollment enroll(EnrollmentRequest req) {
    Enrollment enrollment = new Enrollment();
    enrollment.setUserId(req.getUserId());
    enrollment.setCourseId(req.getCourseId());
    enrollment.setStatus("active");

    enrollmentDao.save(enrollment);

    // Publish cascade event
    eventPublisher.publishEnrollmentCreated(enrollment);

    return enrollment;
}

```

EnrollmentDAO:

```

public void save(Enrollment enrollment) {
    PutItemRequest request = PutItemRequest.builder()
        .tableName("enrollments")
        .item(Map.of(
            "pk", AttributeValue.builder()
                .s(TenantUtil.getTenantCode()) // Auto-scoped
                .build(),
            "sk", AttributeValue.builder()
                .s("ENROLLMENT#" + UUID.randomUUID())
                .build(),
            "userId", AttributeValue.builder().s(enrollment.getUserId()).build(),
            "courseId", AttributeValue.builder().s(enrollment.getCourseId()).build(),
            "status", AttributeValue.builder().s("active").build()
        ))
        .build();

    dynamoDbClient.putItem(request);
}

```

DynamoDB:

- Saves with pk=TENANT#cognizant, sk=ENROLLMENT#uuid
- No cross-tenant visibility

Response:

```

{
  "status": 201,
  "data": {
    "enrollmentId": "...",
    "status": "active"
  },
}

```

```
"error": null
}
```

Q: Design API with token refresh on 401

Frontend (fetchWithAuth):

```
let response = await fetch(url, { headers });

if (response.status === 401) {
  // Refresh token (single call, even if multiple requests fail)
  if (!refreshPromise) {
    refreshPromise = callShellRefresh().finally(() => {
      refreshPromise = null;
    });
  }

  const refreshed = await refreshPromise;
  if (refreshed) {
    // Retry with fresh token
    response = await fetch(url, { headers: getAuthHeaders() });
  }
}

return response;
```

Backend (Lambda Authorizer):

- If token expired: return Deny
- Frontend receives 403 → calls shell.refresh() → retries

KEY INTERVIEW TIPS

1. **Understand DynamoDB:** Single-table design, partition keys, GSI
2. **Tenant isolation:** Every query scoped by tenant (ThreadLocal)
3. **Async patterns:** CompletableFuture, non-blocking I/O
4. **Security:** JWT → Cognito, per-tenant pools, HSTS headers
5. **Caching:** When to cache (30-min TTL), when not (user data)
6. **Error handling:** Custom exceptions, graceful failures
7. **Multi-tenancy:** How isolation prevents data leaks
8. **Performance:** Why async beats sync, exponential backoff

Study this with Frontend and AWS guides for comprehensive coverage